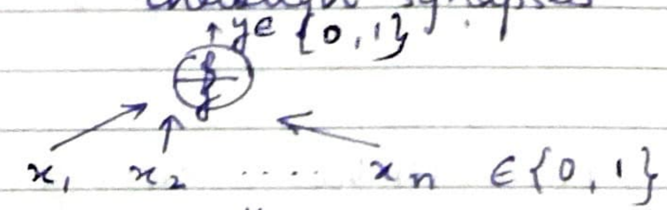
DL - Week-1

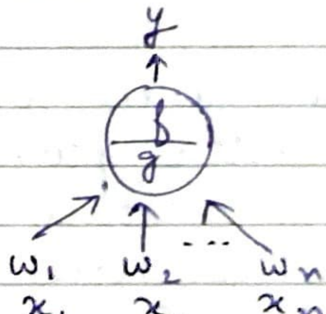
L1.1 History of DL

- single continuous network as opposed to discrete cells → reticular theory + staining technique
- Neuron doctrine → discrete individual cells
- electron microscopy → discrete cells connected through synapses
- MP Neuron



↓

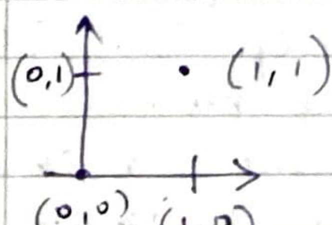
Perceptron



↓

Multi - layered perceptron

- Perceptron limitaⁿˢ → XOR problem (can't work on non-linearly separable funcⁿˢ)



no - line can separate out these 2 classes.

but can be solved by MLP.

- ANNs & Backpropagaⁿ method
  ↳ to learn the wts.

Also, gradient descent → UAT → MLP with enough neurons can approx. any continuous funcⁿ to any desired precision

__/__/__

L1.2 From Cats to ConvNet

- unsupervised pre-training → deep auto-encoder networks to learn a low-dim. representaⁿ of data.

MNIST, Hand-written, speech recog.,
ImageNet (AlexNet → ResNet)

- Cat expt. → Neocogniteron → CNN

L1.3 Faster, Higher, Stronger

- after 2016, faster convergence, better accuracies
  ↳ better optimizaⁿ methods → ADAM Family
  also, better activaⁿ funcⁿˢ → ReLu, etc.
- we have sequences everywhere
  Hopfield → Jordan network ↳ RNNs
- limitaⁿ of RNN → vanishing / exploding gradients problem
- LSTMs → solves vanishing gradient.
- seq.-to-seq models → intro to attenⁿ
- finally, transformers → eg. GPT, BERT
- era of RL agents → beating humans
  ↳ deep RL agents inc. go, chess, poker
- OpenAI Gym → toolkit for RL agents
  ↳ Gym Retro → AlphaStar → MuZero → Player of Games.

__/__/__

L1.4 Madness & the rise of Transformers

- no. of AI papers & patents have increased exponentially.
- rule-based systems → in MT tasks
  ↳ IBM models (Parameter estimaⁿ) → Neural MT (seq2seq models) → transformer models
- BERT → general pre-training & then preparing it for down stream use case (fine tuning)
- today, we have Trillion parameter club.
- from lang. to vision (AlexNet → ResNet)
  ↳ image classificaⁿ
  Object detecⁿ & segmentaⁿ → RNN → YOLO → Transform
- from discriminaⁿ to generation (synthesis of image) ⇒ VAE → GAN → Diffusion-based models → DALL.E 2 (describe to generate)

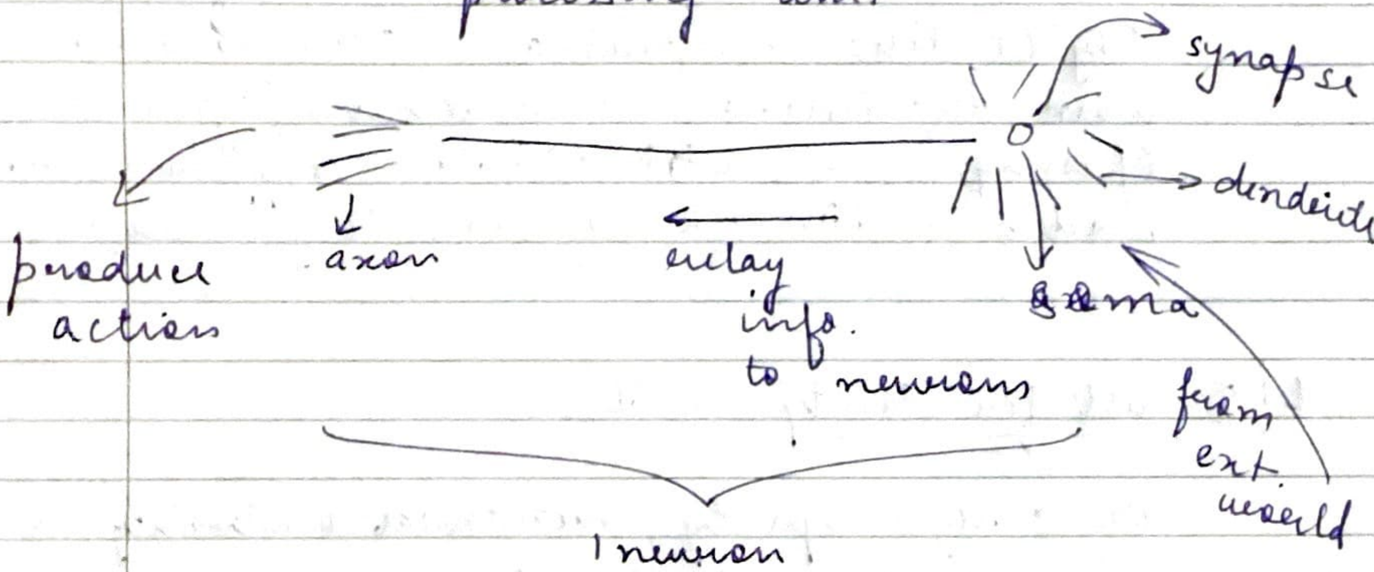
L1.5 Call for Sanity in AI

- DL → high capacity, numerical instability, sharp minima, non-robustness
  ↳ we need explainability & theoretical justificaⁿ
- story of cleverhans
- Black Box NLP → IML (Interpretable ML)
- fair & responsible (because of the biasness in the data used for training)
  ↳ led to EU Regulaⁿ
- push for green AI → call for energy & policy consideraⁿˢ for DL.
- Analog AI → Programmable resistors

- protein-folding problem, astronomy (galaxy evolution, age of galaxy problem)
- AI models for edge-case devices
  constraints → power, storage, standalone, real-time output

L1.6 Motivation from Biological Neuron

- DNN → Artificial Neuron (basic unit)
- biological neurons = neural cells = neural processing unit

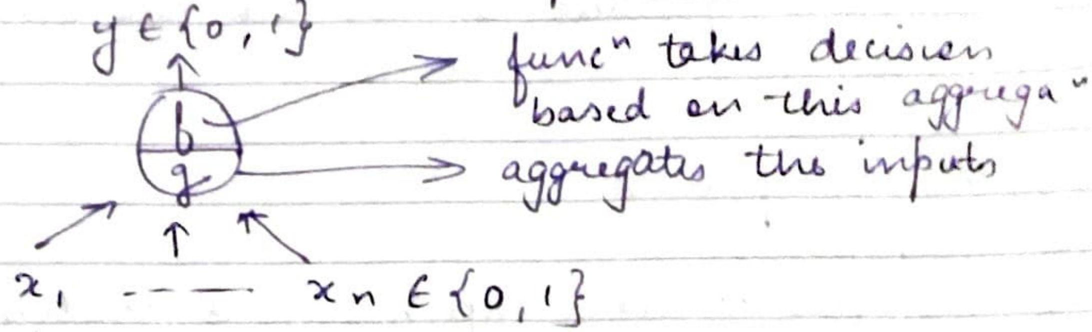


but we have interconnecⁿ of multiple neurons

- but every neuron has its own specific role. Each of them capture just some info. like detect edges → feature groups → high level objects

1.7 MP Neuron & Thresholding Logic

MP Neuron



these inputs can be excitatory or inhibitory (party spoiler input)

It has thresholding logic.

$y = 0$, if any x_i is inhibitory

$$g(x_1, \dots, x_n) = \boxed{g(x) = \sum_{i=1}^{n} x_i}$$

$$y = f(g(x)) = 1 \quad \text{if } g(x) \geq \theta$$

$$= 0 \quad g(x) < \theta$$

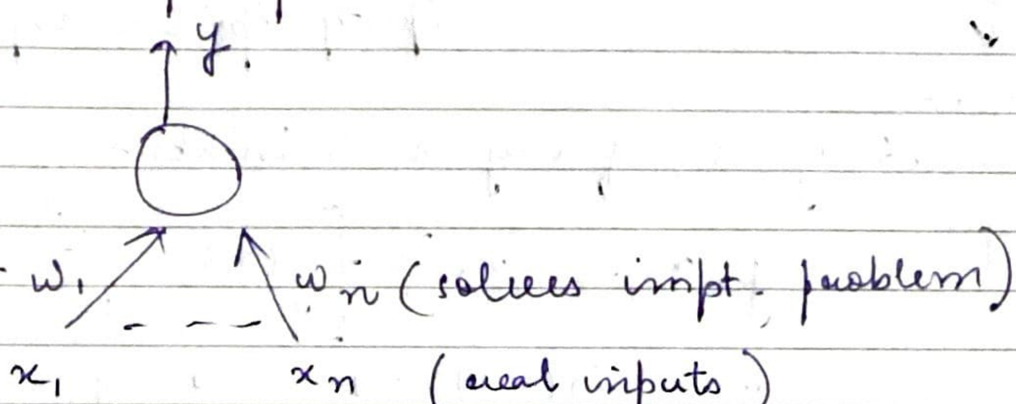
θ → thresholding parameter

- we can implement simple boolean func^{ns} like AND, OR using MP neuron
  - OR ($\theta = 1$)
  - AND (θ = no. of inputs)
- Geometrically, MP neuron draws a line to seperate out the classes (0, 1)
- If we have more than 2 pts. Now, instead of a line, we will get a plane.
- MP Neuron only works on linearly separable func^{ns}.
- Linear separability → There exists a line (or a plane) st. all inputs which

produces a 1 line on one side & all inputs which produce 0 lies on the other side.

L1.8 Perceptrons

- non-boolean inputs, θ handcoded, are all inputs equal? what abt. not linearly sep.?
  ↳ concept of wts. (importance)
- Classical perception



we can learn these wts. based on the given data

$$y = 1 \text{ if } \sum_{i=1}^{n} w_i \times x_i \geq \theta$$
$$= 0 \text{ if } \sum_{i=1}^{n} w_i * x_i < \theta$$

take θ on one side

more accepted conven^n

$$y = 1 \text{ if } \sum_{i=0}^{n} w_i \times x_i \geq 0$$
$$= 0 \text{ if } \sum_{i=0}^{n} w_i \times x_i < 0$$

where, $x_0 = 1$, $w_0 = -\theta$ → bias

- take old task of going to a movie or not
- w_0 → bias, represents the prior (prejudice) $(-\theta)$

- we are still finding linear boundaries only in the perceptron → still works for linearly separable func^{ns}.

L1.9 Error & Error Surfaces

- above & below the line does not necessarily define the half space. It mainly depends on how the eq^n has been formed.
- now, we can think of error as func^n of (w_1, w_2)
- we have to now find an algo. which finds the value of w_1 & w_2 such that it minimizes my error.

L1.10 Perceptron Learning Algo.

- perceptron so far acts as a binary classifier

$x_1 \dots x_n$ (are boolean & real no. inputs)

$w_1 \dots w_n$ (wts. associated with inputs)

algo. converges when all inputs are classified properly.

P → label 1 inputs
N → label 0 inputs
init w randomly
till I reach convergence -
  if $x \in P$ & $\Sigma < 0$ (bad)
    $w = w + x$
  otherwise ($x \in N$ & $\Sigma \geq 0$)
    $w = w - x$

- $w . x = w^T x = \sum_{i=0}^{n} w_i . x_i$

New perceptron rule. $\begin{cases} y = 1 & \text{if } w^T x \geq 0 \\ \quad = 0 & \text{if } w^T x < 0 \end{cases}$

find the line $w^T x = 0$ which divides the input space in 2 halves. Every pt. on this line satisfies eqⁿ $w^T x = 0$.

→ α btw w & any pt. x which lies on the line.

← $\angle$ is $90°$

angle vector w is $\perp$ to the entire line

$w^T x > 0 \quad (\angle < 90°)$ $\qquad \cos\alpha = \frac{w^T x}{\|w\| \|x\|}$

$w^T x < 0 \quad (\angle > 90°)$

- $w = w + x \Rightarrow \cos(\alpha_{new}) > \cos\alpha$

$\Rightarrow \alpha_{new} < \alpha$

$w = w - x \Rightarrow \cos(\alpha_{new}) < \cos\alpha$

$\Rightarrow \alpha_{new} > \alpha$

→ we will do this iteratively.
And the algo: picks pts. randomly. Cycle through entire data & if nothing gets updated that means, I have reached convergence.

4.1.11 Proof of Convergence

- Two sets P & N of pts. in an n-dim space are called absolutely linearly sep. if n+1 real nos. $w_0 \ldots w_n$ exist s.t. every pt-

__/__/__

$(x_1 \ldots x_n) \in P$ satisfies $\Sigma \geq w_0$ & every pt. $(x_1 \ldots x_n) \in N$ satisfies $\Sigma < w_0$

- If N & P are finite sets & linearly sep. the perceptron learning algo. updates the wt. vector w only a finite no. of times.
- one new approach

  P, N, N^- (negaⁿ of all pts. in N)

  $P' = P \cup N^-$

  now everything is positive in our set

  $p \in P'$ → $p = \frac{p}{\|p\|}$ $(\|p\| = 1)$

  if $\Sigma < 0$ then,

  $w = w + p$

  ⇒ just 1 update is totally enough to converge the entire dataset.

we want all pts. in P' to lie in the half-space.

no other if condⁿ is reqd.

we know that one w^* exists, and we have linearly seperable data → optimal soluⁿ, we still don't what it is.

we just req. finite k no. of updates to reach the convergence.